ADITYA UNIVERSITY

School of Computing

Department of Artificial Intelligence and Machine Learning

Object Oriented Programming through C++

CERTIFICATE

This is to certify that the Cornerstone project work entitled *"LIBRARY MANAGEMENT SYSTEM"*. A C++ - based Libarary Operations, Analysis and Computation System is submitted by:

MD.ABIDA	- (25B11AI747)
M. NAVYA SANTHOSHINI	- (26B21AI076)
N.TANUJA	- (25B11AI786)
CH.BHAVANA	- (26B21AI130)
D.NANDU	- (25B11AI270)

In partial fulfilment of the requirements for the award of the B.Tech. degree in Artificial intelligence and machine learning the academic year 2026-2027.

Aditya University, Aditya Nagar, ADB Road, Surampalem, Kakinada, District, Andhra Pradesh, India - 533437

CERTIFICATE

This is to certify that the Cornerstone Project entitled “Library Management System” is submitted by the students of the Department of Artificial Intelligence and Machine Learning, Aditya University, in partial fulfilment of the academic requirements for the B.Tech. degree.

The project has been developed as part of the course Object Oriented Programming through C++. The work presented in this documentation describes the planning, implementation, testing and current scope of the Library Management System developed by the project team.

The project work has been carried out under the prescribed academic guidance and is presented for evaluation as a Cornerstone Project.

Project Guide

T. Vijaya Lakshmi,

Department of AIML,

Aditya University .

DECLARATION

We hereby declare that the project entitled “Library Management System” is our original academic work carried out as part of the Cornerstone Project for Object Oriented Programming through C++ at Aditya University.

We further declare that this project has been prepared for academic evaluation and has not been submitted, either in full or in part, for the award of any other degree or diploma. The work has been carried out with academic honesty and with appropriate acknowledgement of the resources used during development.

We accept responsibility for the accuracy of the project information, implementation details and documentation presented in this report.

SUBMITTED BY

MD.ABIDA - (25B11AI747)

M.NAVYA SANTHOSHINI - (26B21AI076)

N.TANUJA - (25B11AI786)

CH.BHAVANA - (26B21AI130)

D.NANDU - (25B11AI270)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **T. Vijaya Lakshmi , Assistant Professor,** who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department**, Dr. R. Suneetha , Assistant Professor, Dr. Bollu Manikyala Rao, Assossicate Professor** and **Dr. Dasari Veeraswamy, Assistant Professor**, Department of AIML, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, **Dr. Kovvuri N Bhargavi, Associate Professor & HOD**, for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks **to Dr. M.V. Rajesh, Associate Dean, School of Computing**, for his encouragement and support during our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar; Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences); Dr. S. Rama Sree, Pro Vice-Chancellor (Academics); Dr. M.B. Srinivas, Vice-Chancellor; and Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management**, **Aditya University,** for providing excellent infrastructure and state-of-the-art laboratories

Finally, we would like to thank the faculty members, lab technicians, nonteaching staff, and our friends who have directly or indirectly supported us in completing this Cornerstone project successfully.

ABSTRACT

The Library Management System is an educational application developed using C++ as part of the Object-Oriented Programming course. The main purpose of the system is to provide a simple and efficient platform for managing library activities, including book records, member registration, searching, issuing, and returning books.

The system maintains book details such as book ID, title, author, category, and availability status. It also stores member information, including member ID, name, and contact details. Users can search for books by ID, title, or author and check whether a book is available or already issued.

The application provides features for adding and displaying books, registering members, searching member details, issuing books, returning books, and checking book availability. It uses C++ concepts such as classes, structures, vectors, and functions to organize the program efficiently.

File handling is used to save and retrieve book and member records through text files. Input validation helps prevent duplicate IDs and invalid operations, making the system more reliable and easier to manage.

The system offers a simple and organized solution for library management. Future enhancements may include fine calculation, due-date tracking, and graphical user interface integration.

ABBREVIATIONS

Abbreviations	Description
OOP	Object-Oriented Programming
C++	C Plus Plus
LMS	Library Management System
STL	Standard Template Library
ID	Identification Number
I/O	Input / Output
GUI	Graphical User Interface
CLI	Command Line Interface
IDE	Integrated Development Environment
CSE	Computer Science and Engineering

LIST OF FIGURES

Figure 1. High-Level System Architecture

Figure 2. Library Management System Flowchart

Figure 3. OOP Design of the Library Class

Figure 4. Book Management Module

Figure 5. Main Menu of the Library Management System

Figure 6. Book Availability Output

Figure 7 Book Addition Output

LIST OF TABLES

Table 1. Current Scope

Table 2. Hardware and Software Requirements

Table 3. Book Record Structure

Table 4. Member Record Structure

Table 5. Validation and Error Handling

TABLE OF CONTENTS

CONTENTS	PAGE NO
Title Page	I
Certificate	II
Declaration	III
Acknowledgement	IV
Abstract	V
Abbreviations	VI
List of Figures	VII
List of Tables	VIII
Table of Contents	IX
CHAPTER – 1 PROJECT PLANNING	1
1.1 Problem Statement	1
1.2 Project Objectives	2
1.3 Planned Modules	3–4

1.4 System Architecture 5

1.5 Algorithms and Flowchart 6

CHAPTER – 2 PROJECT IMPLEMENTATION 7

2.1 OOP Concepts Used for Implementation 7

2.2 Technology Stack and Project Structure 8

2.3 Library Class and Core Functions 9–10

2.4 Book Management Operations 11

2.5 Member Management Operations 12

2.6 Book Issue and Return Operations 13

2.7 File Handling and Data Storage 14

2.8 Validation and Error Handling 15

2.9 Testing and Outputs 16–18

CHAPTER – 3 CONCLUSION & FUTURE SCOPE 19

3.1 Conclusion 19

3.2 Future Scope 20

References 21

CHAPTER-1

1.1 Problem Statement

Library management is an important activity in educational institutions, colleges, and public libraries. Managing book records, member information, and book issuing and returning manually can be time-consuming and may lead to errors.

A common difficulty in library management is maintaining accurate records of books and members. Searching for a particular book, checking its availability, and managing issued books can become difficult as the number of records increases.

The Library Management System addresses these challenges by providing a simple application developed using C++. It allows users to add and display books, register members, search records, issue and return books, and check book availability. The application also uses file handling to save and retrieve book and member information.

Project Scope

• The system allows users to add and display book records.

• The application supports searching books by ID, title, or author.

• The system provides member registration and member search functionality.

• Users can issue available books to registered members.

• The application supports returning issued books and checking availability.

• Book and member records are stored in text files for future use.

• The system validates IDs and prevents duplicate records and invalid operations.

Table 1. Current Scope

Area	Current Scope
Book Management	Adding, displaying, and searching book records
Member Management	Registering, displaying, and searching members

Book Circulation	Issuing and returning books
Availability	Checking whether books are available or issued
Backend	C++ classes, structures, vectors, and functions
File Handling	Storing records in books.txt and members.txt
Validation	Preventing duplicate IDs and invalid operations

1.2 Project Objectives

The project is designed with both educational and implementation objectives. The educational objectives focus on understanding library management concepts in a structured manner. The implementation objectives focus on applying C++ programming concepts to develop a simple library management application.

• To understand the importance of library management and book organization.

• To maintain book details such as ID, title, author, and category.

• To register members and manage their information.

• To implement book searching using ID, title, and author.

• To implement book issuing and returning operations using C++.

• To check book availability and maintain issue status.

• To demonstrate classes, structures, vectors, and functions.

• To store and retrieve records using text files.

• To validate inputs and prevent duplicate book and member IDs.

• To provide a modular foundation for future enhancements.

1.3 Planned Modules

The application is organized into clearly defined modules so that library operations remain easy to understand and manage. Each module performs a specific task, including book management, member registration, searching, issuing, returning, and data storage.

Book Management

This module allows users to add new books and display book records. It maintains book ID, title, author, category, and issue status.

Member Management

This module allows users to register new library members, display member details, and search for members using their unique IDs.

Book Search

This module allows users to search for books using book ID, title, or author name. It displays matching records along with their availability status.

Book Issue and Return

This module manages book circulation. It allows users to issue available books to registered members and return books that have already been issued.

File Handling

This module saves and retrieves book and member records using books.txt and members.txt, preserving data between program executions.

1.4 System Architecture

The Library Management System follows a modular architecture in which the C++ application manages book records, member information, and library operations. The user interacts with the application through a menu-driven console interface.

The Library class acts as the central component of the system. It provides functions for adding and displaying books, registering and searching members, issuing and returning books, and checking book availability. Structures are used to represent book and member records, while vectors maintain the records during program execution.

File handling is used to save and retrieve information from books.txt and members.txt. Input validation checks IDs and prevents duplicate records and invalid operations, helping the system manage library data reliably.

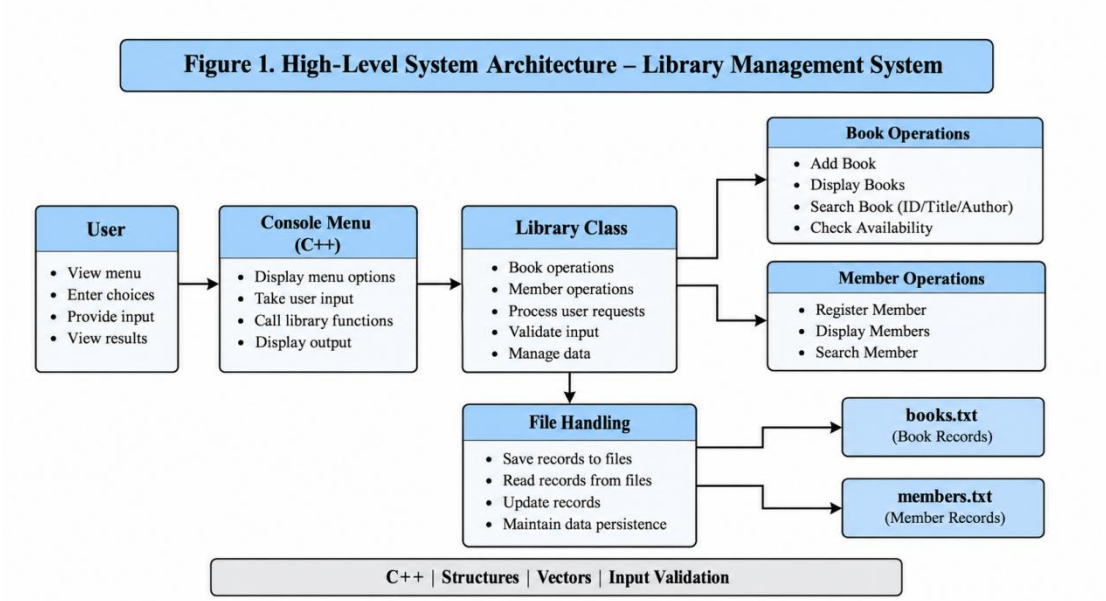


Figure 1. High-Level System Architecture

System Flow

User → Console Menu → Library Class → Book and Member Operations → Text File Storage → Result Display

Figure 1. High-Level System Architecture

Architecture Responsibilities

• The console menu accepts user choices and displays results.

• The Library class manages book and member operations.

• Structures and vectors organize book and member records.

• File handling saves and loads records from text files.

• Input validation prevents duplicate IDs and invalid operations.

1.5 Algorithms and Flowchart

The Library Management System uses a menu-driven algorithm to perform different library operations. The program loads existing records, displays the main menu, accepts the user's choice, performs the selected operation, and returns to the menu until the user chooses to exit.

Book Addition Algorithm

• Start the operation.

• Enter the book ID.

• Check whether the ID is valid and unique.

• Enter the book title, author, and category.

• Validate the required book details.

• Add the book to the records.

• Save the updated records to books.txt.

• Display a success message.

Book Search Algorithm

• Enter the book ID, title, or author name.

• Compare the search value with stored book records.

• Display matching book details and availability status.

• If no match is found, display an appropriate message.

• Return to the main menu.

Book Issue Algorithm

• Enter the book ID and member ID.

• Verify that the book and member exist.

• Check whether the book is available.

• Update the issue status and member ID.

• Save the updated book records.

• Display the result.

Book Return Algorithm

• Enter the book ID.

• Verify that the book exists and is currently issued.

• Update the book status to available.

• Reset the associated member ID.

• Save the updated records.

• Display a success message.

Book Availability Algorithm

• Enter the book ID.

• Search for the book in the records.

• Check its current issue status.

• Display whether the book is available or issued.

• Return to the main menu.

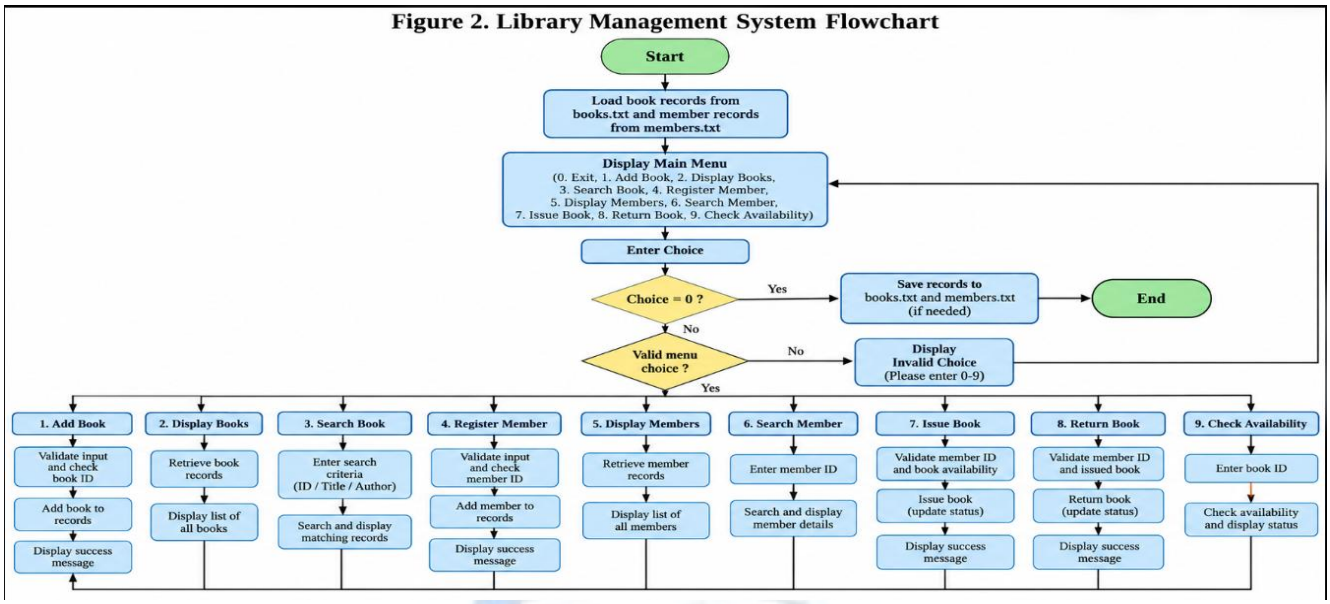


Figure 2. Library Management System Flowchart

Insert the flowchart showing the start, loading records, main menu, input validation, book and member operations, and exit process.

CHAPTER – 2 PROJECT IMPLEMENTATION

Object-Oriented Programming (OOP) is used in the C++ implementation of the Library Management System to organize book records, member information, and library operations. The project uses the `Library` class, `Book` and `Member` structures, vectors, member functions, and file handling to manage library activities systematically.

Classes and Objects

The `Library` class manages the main operations of the system, including adding books, displaying records, searching for books, registering members, issuing books, and returning books. An object of the `Library` class is used to access these operations.

Structures

The project uses two structures, `Book` and `Member`, to represent library records. The `Book` structure stores details such as book ID, title, author, category, and availability status. The `Member` structure stores member ID, name, and contact details.

Encapsulation

The `Library` class groups related library operations and record-management functionality in one place. This helps organize the program and makes its functions easier to maintain.

Member Functions

Member functions perform specific tasks, such as adding and displaying books, searching for book and member details, issuing and returning books, and checking book availability.

Vectors

C++ vectors are used to maintain collections of book and member records while the program is running. They allow the system to manage multiple records efficiently.

File Handling

The project uses `books.txt` and `members.txt` to save and retrieve book and member information. This allows records to be preserved between program executions.

Input Validation

The application validates important inputs by checking for duplicate IDs, missing records, and invalid operations. These checks help maintain accurate records and prevent common errors.

2.2 Technology Stack and Project Structure

The Library Management System is developed using C++ as a menu-driven console application. The system provides features for managing book records, registering members, searching for books and members, issuing and returning books, and checking book availability. C++ classes, structures, vectors, and functions are used to organize the application logic. File handling is used to store and retrieve records through text files.

The application uses a console-based interface to accept user input and display results. The `Library` class manages the main library operations, while the `Book` and `Member` structures represent book and member information. The `books.txt` and `members.txt` files provide persistent storage for the records.

Table 2. Technology Stack

Layer	Technology	Purpose
Programming Language	C++	Implements library operations and program logic
User Interface	Console Menu	Accepts user choices and displays results
Data Representation	Structures	Stores book and member details
Data Management	Vectors	Maintains book and member records during execution
Object-Oriented Design	Classes and Objects	Organizes library operations
Program Organization	Functions	Performs specific tasks such as searching, issuing, and returning books
File Storage	Text Files	Stores book and member records
File Handling	C++ File Streams	Reads and writes records to text files
Input Validation	C++ Conditions	Checks duplicate IDs, missing records, and invalid operations
Development Environment	C++ Compiler and IDE	Compiles and executes the program

Project Directory Structure

The project contains the main C++ source file and two text files used for data storage.

```
Library-Management-System/
│
├── libsys.cpp
├── books.txt
└── members.txt
```

- libsys.cpp: Contains the main program, `Library` class, `Book` and `Member` structures, and library operation functions.
- books.txt: Stores book records, including relevant book details and availability information.
- members.txt: Stores registered member information.

This project structure keeps the program logic separate from the stored records and provides a simple approach to managing library information.

2.3 Library Class and Core Functions

The Library class is responsible for managing book records, member information, and library operations. The project uses C++ structures to represent books and members, while vectors maintain records during program execution. The class provides functions for adding, displaying, and searching books and members, issuing and returning books, and checking book availability.

Book Structure

The Book structure stores information about each book, including book ID, title, author, category, and availability status. These details help identify books and manage their circulation.

Member Structure

The Member structure stores member ID, name, and contact details. It maintains registered member information and supports book issuing operations.

Library Class Functions

addBook()

Adds a new book record to the library after validating the required details.

displayBooks()

Displays the book records stored in the system.

searchBook()

Searches for books using the book ID, title, or author.

registerMember()

Registers a new library member and stores the member details.

displayMembers()

Displays the registered member records.

searchMember()

Searches for a member using the member ID.

issueBook()

Issues an available book to a registered member after checking the required records.

returnBook()

Updates the book's status when an issued book is returned.

checkAvailability()

Checks whether a particular book is available or already issued.

File Handling Functions

Load and save book and member records using the text files books.txt and members.txt.

OOP Design Diagram

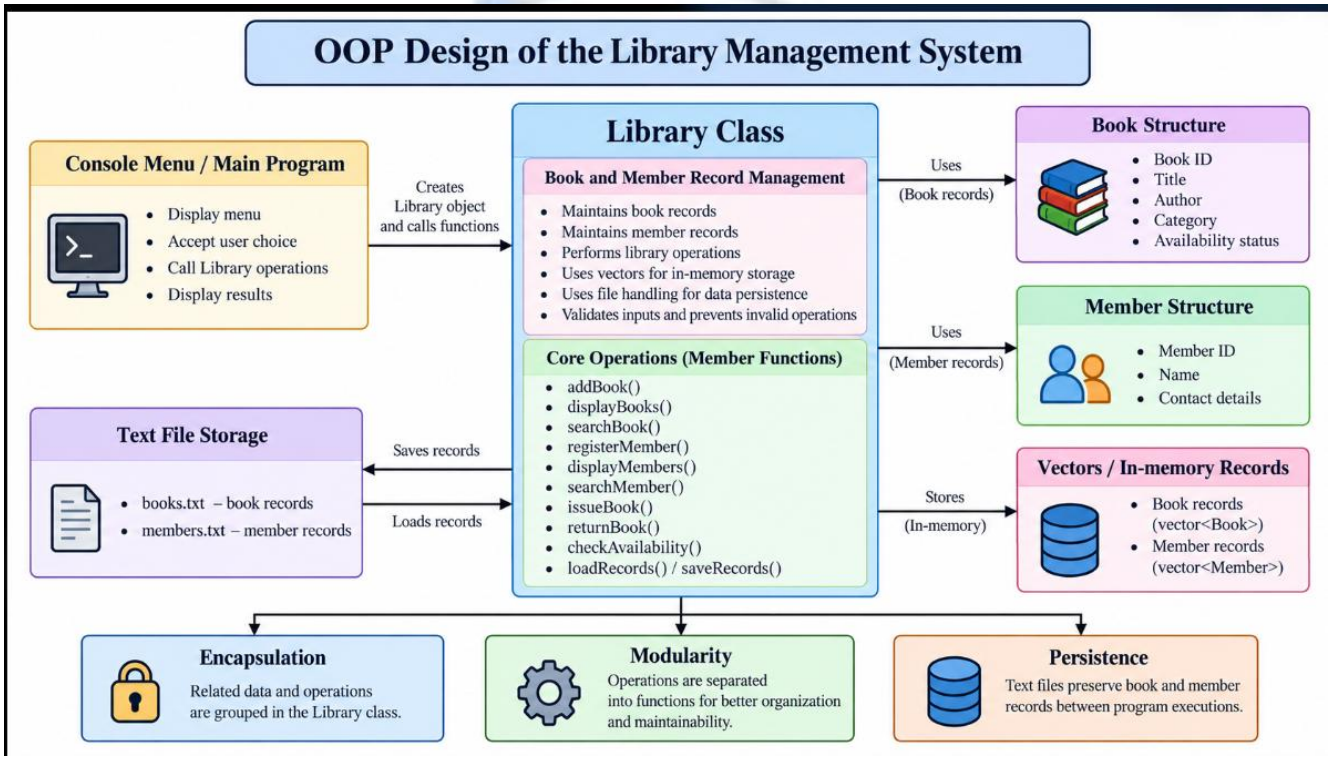


Figure 3. OOP Design of the Library Management System

The diagram illustrates the relationship between the console menu, Library class, Book and Member structures, vectors, and text-file storage. It also shows how library operations are organized and how records are preserved between program executions.

2.4 Book Management Operations

The Book Management Module is responsible for adding, displaying, and searching book records. The system stores book details such as book ID, title, author, category, and availability status. It validates the entered information and prevents duplicate book IDs.

Adding Books

The book addition operation accepts the book ID, title, author, and category. The system checks whether the ID already exists before adding a new record.

```
void addBook() {
    // Accept book details
    // Validate the book ID
    // Check for duplicate IDs
    // Store the new book record
    // Save updated records
}
```

Displaying Books

The system displays the stored book records, including their IDs, titles, authors, categories, and availability status. This allows users to view the books maintained by the library.

Searching Books

The search operation allows users to locate books using the book ID, title, or author. If a matching record is found, the system displays its details. Otherwise, it displays an appropriate message.

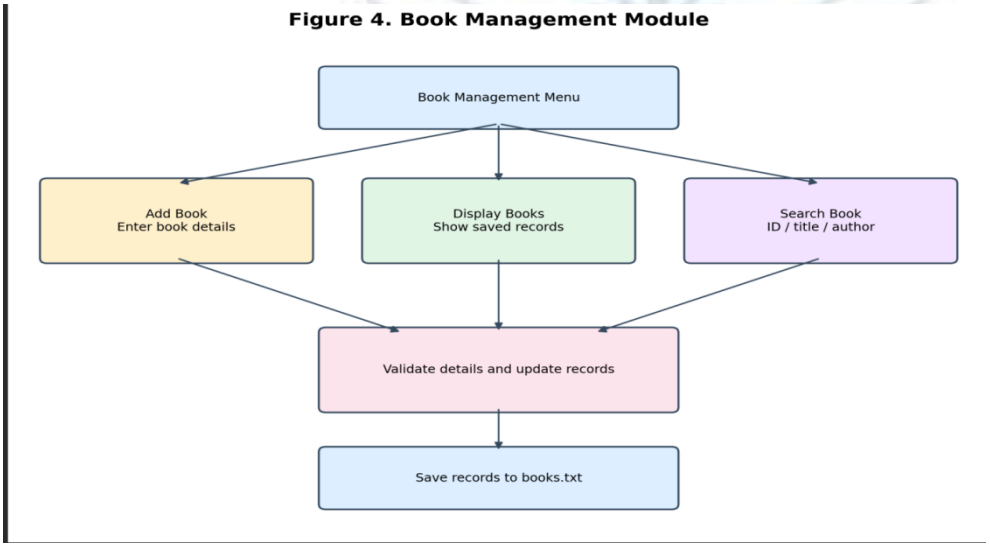


Figure 4. Book Management Module

2.5 Member Management Operations

The Member Management Module maintains information about registered library members. It supports member registration, displaying member records, and searching for members.

Member Registration

The user enters the member ID, name, and contact details. The application checks the entered information and prevents duplicate member IDs.

Displaying Members

The system displays registered member information so that users can view the records maintained by the library.

Searching Members

The user enters a member ID to search for a registered member. If a matching record exists, the application displays the member details.

2.6 Book Issuing and Returning Operations

The Book Issuing and Returning Module manages the circulation of books. It checks book and member records and updates the availability status when a book is issued or returned.

Book Issuing

The user enters the book ID and member ID. The application verifies that the book and member exist and checks whether the book is available. If the conditions are satisfied, the system issues the book and updates its status.

Book Returning

The user enters the ID of the book being returned. The system verifies that the book exists and is currently issued. If the return is valid, the book status is updated to available.

Book Availability

The availability operation checks the status of a selected book and displays whether it is available or issued.

2.7 File Handling and Data Storage

File handling is used to preserve book and member records between program executions. The application uses text files to store the information and retrieve it when required.

Book File Storage

The `books.txt` file stores book records. The application reads saved information and writes updated book records when required.

Member File Storage

The `members.txt` file stores registered member information. This allows member records to be maintained between program executions.

Loading and Saving Records

When the application starts, existing records can be loaded into the program. After relevant changes, the updated records are saved to the corresponding text files.

2.8 Validation and Error Handling

Validation is performed to prevent invalid library operations and maintain accurate records. The application checks input values and verifies the current state of records before processing requests.

Table 3. Validation and Error Handling

Condition	Validation Rule	Expected Result
Duplicate book ID	Check whether the book ID already exists	Reject duplicate record
Duplicate member ID	Check whether the member ID already exists	Reject duplicate registration
Empty book details	Validate required fields	Display an error message
Book not found	Search using the entered book ID	Display a not-found message
Member not found	Search using the entered member ID	Display a not-found message

Book already issued	Check book availability	Prevent another issue
Invalid return	Check whether the book is currently issued	Reject invalid return

Input Validation

The application checks important inputs before performing library operations. This helps prevent duplicate records and operations involving missing or invalid records.

Error Messages

The system displays appropriate messages when a book or member cannot be found, an ID is duplicated, or an operation cannot be completed.

2.9 Outputs

The Library Management System displays the results of different operations through a console interface. It provides information about book records, book availability, and the status of library operations.

Main Menu Output

The main menu displays the available operations, including adding books, displaying all books, searching books, registering members, displaying and searching members, issuing and returning books, and checking book availability.

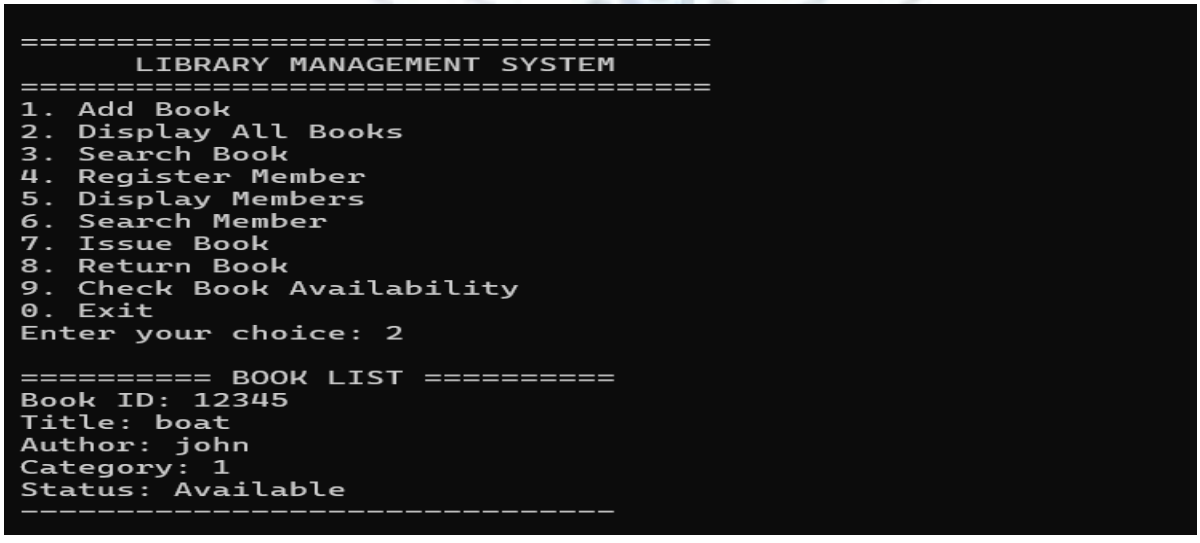


Figure 5. Main Menu of the Library Management System

Book Availability Output

The system allows users to check whether a particular book is available or issued. The output displays the book ID, title, and current status.

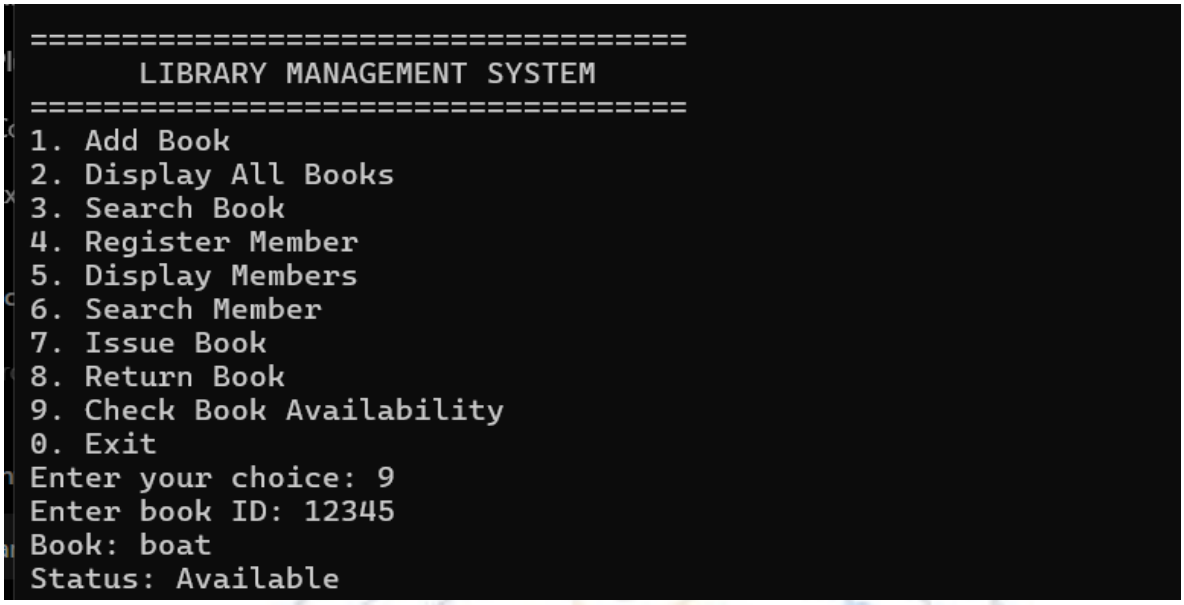


Figure 6. Book Availability Output

Book Addition Output

The application allows users to add new books by entering the book ID, title, author name, and category. After successful validation, the system displays a confirmation message.

```
==================================
      LIBRARY MANAGEMENT SYSTEM
==================================
1. Add Book
2. Display All Books
3. Search Book
4. Register Member
5. Display Members
6. Search Member
7. Issue Book
8. Return Book
9. Check Book Availability
0. Exit
Enter your choice:
1
Enter book ID: 76271261
Enter book title: Blue
Enter author name: Greshan
Enter category: Comedy
Book added successfully.
```

Figure 7 Book Addition Output

These outputs demonstrate the basic operations of the Library Management System through its console interface.

CHAPTER – 3 CONCLUSION & FUTURE SCOPE

3.1 Conclusion

The Library Management System provides a simple and organized solution for managing library activities. The project allows users to add and display books, search for book details, register members, search member records, issue and return books, and check book availability through a menu-driven console interface.

The project demonstrates the application of C++ programming concepts in developing a practical system. The Library class organizes library operations, while the `Book` and `Member` structures represent book and member information. Vectors are used to maintain records during program execution, and functions separate the different operations into manageable components.

The project also demonstrates file handling for data storage. The `books.txt` and `members.txt` files are used to save and retrieve book and member records, allowing the information to be maintained between program executions.

Input validation helps prevent duplicate IDs, missing-record operations, and invalid book issuing or returning requests. These features improve the reliability and usability of the application.

The modular structure of the Library Management System provides a foundation for future improvements, such as fine calculation, due-date tracking, report generation, and database integration.

Key Outcomes

The system supports book addition, display, and search operations.

The system provides member registration, display, and search functionality.

The system supports issuing and returning books.

The system allows users to check book availability.

C++ classes, structures, vectors, and functions are used to organize the implementation.

File handling provides storage for book and member records.

Input validation helps prevent duplicate records and invalid operations.

3.2 Future Scope

The Library Management System can be enhanced by introducing additional features to improve its functionality and make library management more efficient.

Introduce due-date tracking for issued books.

Implement automatic fine calculation for overdue books.

Add book categories and advanced search filters.

Generate reports for available, issued, and returned books.

Integrate a database management system for storing larger volumes of records.

Develop a graphical user interface to improve user interaction.

Add user authentication and role-based access for librarians and members.

Provide book reservation and renewal functionality.

Implement email or notification reminders for return dates.

Expand input validation and automated testing to cover additional cases.

Recommended Development Direction

Future development can proceed incrementally. Due-date tracking and fine calculation can first be added to the book circulation module. Database integration and a graphical user interface can then be introduced to improve record management and usability. Additional features, such as reservations, renewals, and notifications, can be implemented as the system evolves.

Limitations of the Current Version

The current application is a console-based C++ program and does not provide a graphical user interface.

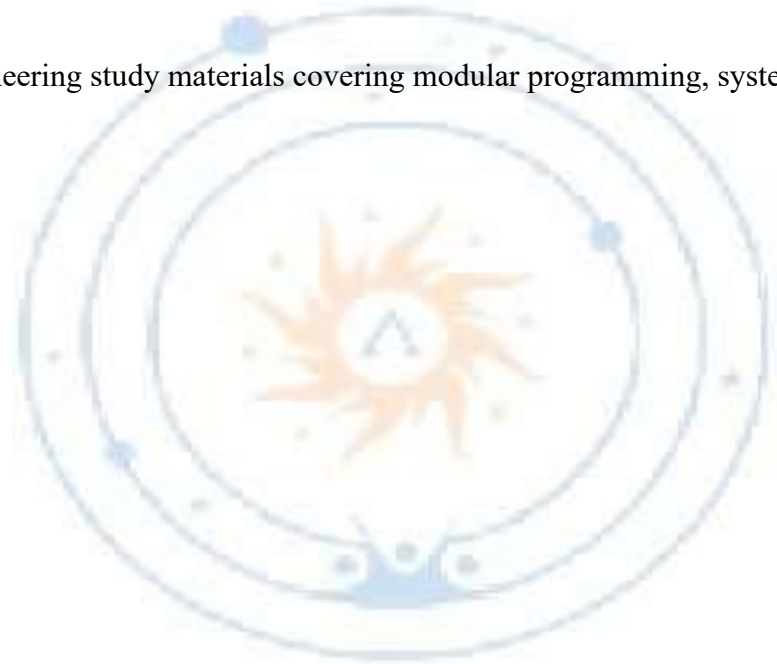
The application uses text files for storing book and member records rather than a database management system.

The current implementation does not include automatic fine calculation or due-date reminders.

Online book reservations, renewals, and notification services are not included in the current version.

REFERENCES

- Library Management System project source code – `libsys.cpp`.
- C++ documentation covering classes, structures, functions, vectors, and Object-Oriented Programming concepts.
- C++ Standard Library documentation for file input/output operations and file streams.
- C++ reference materials for input validation, conditional statements, and exception handling.
- Course materials and classroom notes on Object-Oriented Programming using C++.
- C++ compiler and development environment documentation used for compiling and executing the application.
- Software engineering study materials covering modular programming, system design, and error handling.



C++ SOURCE CODE

1. Frontend

frontend/main.cpp

Purpose: Provides the menu-driven user interface and accepts user choices.

```
#include <iostream>
#include "../backend/Library.h"

using namespace std;

int main() {
    Library library;
    int choice;

    do {
        cout << "\n================================\n";
        cout << "   LIBRARY MANAGEMENT SYSTEM\n";
        cout << "================================\n";
        cout << "1. Add Book\n";
        cout << "2. Display Books\n";
        cout << "3. Search Book\n";
        cout << "4. Register Member\n";
        cout << "5. Display Members\n";
        cout << "6. Search Member\n";
        cout << "7. Issue Book\n";
        cout << "8. Return Book\n";
        cout << "9. Check Availability\n";
        cout << "0. Exit\n";
        cout << "Enter your choice: ";
```

```
        cin >> choice;

        switch (choice) {
            case 1: library.addBook(); break;
            case 2: library.displayBooks(); break;
            case 3: library.searchBook(); break;
            case 4: library.registerMember(); break;
            case 5: library.displayMembers(); break;
            case 6: library.searchMember(); break;
            case 7: library.issueBook(); break;
            case 8: library.returnBook(); break;
            case 9: library.checkAvailability(); break;
            case 0: cout << "Exiting system.\n"; break;
            default: cout << "Invalid choice.\n";
        }

    } while (choice != 0);

    return 0;
}
```

2. Backend

backend/Library.h

Purpose: Declares the `Book` and `Member` structures and the `Library` class functions.

```
#ifndef LIBRARY_H
#define LIBRARY_H

#include <iostream>
#include <fstream>
#include <vector>
```

```
#include <string>

using namespace std;

struct Book {
    int id;
    string title;
    string author;
    bool issued;
    int issuedTo;
};

struct Member {
    int id;
    string name;
    string phone;
};

class Library {
private:
    vector<Book> books;
    vector<Member> members;

    void loadBooks();
```

```
    void saveBooks();
    void loadMembers();
    void saveMembers();

    int findBook(int id);
    int findMember(int id);

public:
    Library();

    void addBook();
    void displayBooks();
    void searchBook();

    void registerMember();
    void displayMembers();
    void searchMember();

    void issueBook();
    void returnBook();
    void checkAvailability();
};

#endif
```

3. Data Storage

data/books.txt

Stores book IDs, titles, authors, issue status, and borrower IDs.

data/members.txt

Stores member IDs, names, and phone numbers.

4. Build Configuration

```
project(LibraryManagementSystem)

set(CMAKE_CXX_STANDARD 11)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(LibraryManagementSystem
    frontend/main.cpp
    backend/Library.cpp
)
```